

Symmetric Programming and Reasoning

Ohad Kammar, LFCS, University of Edinburgh



joint work with Matija Pretnar, FMF, University of Ljubljana
Workshop in Celebration of
Marcelo Fiore's 60th Birthday (FioreFest)
3 July 2026



THE UNIVERSITY OF EDINBURGH
informatics **lfcs**

Laboratory for Foundations
of Computer Science



supported by:



Advanced
Research
+ Innovation
Agency

ARIA

Symmetric reasoning without-loss-of-generality

Theorem (small pigeonhole principle¹)

If three balls are painted red or blue, at least two have the same color.

Proof

Without loss-of-generality, ball 1 is blue. If balls 2 and 3 are the same colour, pick them and conclude. If they have different colours, **without loss-of-generality**, ball 2 is blue. Pick balls 1 and 2 and conclude. ■

Valid proof, but dubious nesting of wlog.

Dijkstra [EWD1223, '95]: typical wlog proofs omit relevant details.

Presents a wlog principle for predicates over pairs.

Harrison ['09]: there ought to be a wlog principle.

Presents 6 wlog HOL light tactics in different domains (goal: formalisation fidelity)

¹Adapted from the Wikipedia entry *wlog*.

Symmetric reasoning without-loss-of-generality

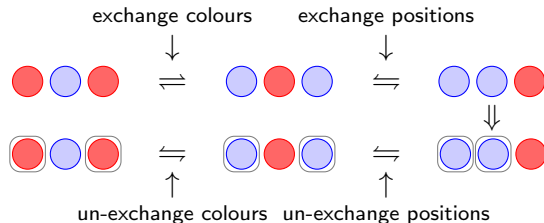
Theorem (small pigeonhole principle¹)

If three balls are painted red or blue, at least two have the same color.

Proof

Without loss-of-generality, ball 1 is blue. If balls 2 and 3 are the same colour, pick them and conclude. If they have different colours, **without loss-of-generality**, ball 2 is blue. Pick balls 1 and 2 and conclude. ■

today: symmetric wlog as group-action **conjugation** of **core constructions**

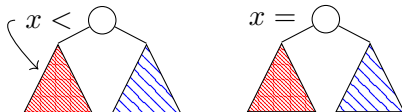


¹Adapted from the Wikipedia entry *wlog*.

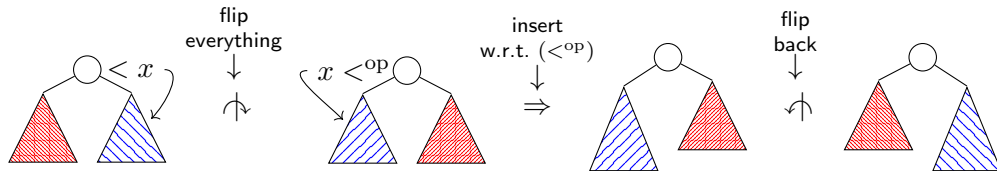
Goal: exploit **symmetry** for **computation** and **reasoning**

Example: binary search tree insertion using **symmetry**

Interesting cases

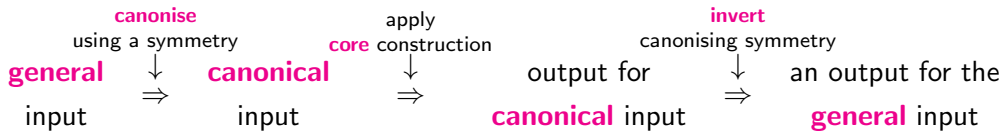


Symmetric case



NB: Yes, silly performance. We'll improve this later.

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** construction)



symmetry = **group** + **action**

Symmetric programming

Exploit symmetries for computation and reasoning—**avoid symmetric cases**.

Mathematical foundations for symmetric programming semantics:

- ▶ **Abstractions** for symmetric programming
- ▶ **Simply-typed** reasoning about equivariance
- ▶ **Dependently-typed** and mathematical reasoning
- ▶ User-defined **data-types** through initial algebra semantics and induction principles
 - ▶ Simply-typed data-types
 - ▶ Dependently-typed/indexed data-types
(Extended exercise in Marcelo-ism)

Talk structure

- ▶ Introduction
- ▶ Simply-typed symmetric programming
- ▶ Symmetric datatypes à la Marcelo
- ▶ Conclusion

Basic group theory

Group action $\langle G, A \rangle$

Group $G = \langle \underline{G}, (*), e, (-)^{-1} \rangle$

G-action $A = \langle \underline{A}, \overset{A}{*} \rangle$

Notation

- **symmetries**: $\underline{G}$ elements
- x **symmetric to** y **via** g when: $g \circledast x = y$

action and group axioms mean:

‘**being symmetric**’ is a
proof-relevant equivalence relation tracked by
symmetries

Rotations $\overset{G:=}{\langle \underline{C}_4, \overset{=:A}{\text{Dir}} \rangle}$

$\underline{C}_4 := \{ \curvearrowleft, \curvearrowright, \curvearrowright, \text{Q} \}$

$\curvearrowleft * \curvearrowleft := \curvearrowright \quad \curvearrowleft * \curvearrowright := \curvearrowright$
 $\curvearrowleft * \curvearrowright := \text{Q} \quad \curvearrowleft * \text{Q} := \curvearrowleft$

acting on

$\underline{\text{Dir}} := \{ \Leftarrow, \Rightarrow, \Uparrow, \Downarrow \}$

$\curvearrowleft \circledast \Leftarrow := \Downarrow \quad \curvearrowleft \circledast \Downarrow := \Rightarrow$
 $\curvearrowleft \circledast \Rightarrow := \Uparrow \quad \curvearrowleft \circledast \Uparrow := \Leftarrow$

laws determine the rest

w.l.o.g. anatomy: **canonisation**

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** construction)

Canoniser for G -action A

function $c : \underline{A} \rightarrow \underline{G}$ assigning **canonising symmetries** to A -elements.

Canonical elements

$$\text{Can } c := \{ (c a) \otimes a \mid a \in \underline{A} \}$$

Example: $\text{mkLft} : \underline{\text{Dir}} \rightarrow \underline{\text{C}}_4$

Canonising symmetry makes direction 'left':

$$\text{mkLft} \Rightarrow := \curvearrowright$$

$$\text{mkLft} \Uparrow := \curvearrowleft$$

$$\text{mkLft} \Leftarrow := \circlearrowleft$$

$$\text{mkLft} \Downarrow := \curvearrowright$$

Only canonical direction is 'left':

$$\text{Can } \text{mkLft} = \{ \Leftarrow \}$$

w.l.o.g. = (input & output **symmetries**) + (**canonising** symmetries) + (**core** construction)

Core function $f : \text{Can } c \rightarrow \underline{B}$

work on canonical elements, typically more concrete, after eliminating symmetric cases

Symmetric programming **abstraction**

w.l.o.g. = (input & output **symmetries**)+(**canonising** symmetries)+(**core** construction)

W.l.o.g. construction

Formally:

$$\frac{G\text{-actions } A, B \quad c : \underline{A} \rightarrow \underline{G} \quad f : \text{Can } c \rightarrow \underline{B}}{\text{wlog}(c, f) := \left(a \mapsto (ca)^{-1} \otimes f((ca) \otimes a) \right) : \underline{A} \rightarrow \underline{B}}$$

Equivariant where defined core functions

Assume an equivariant $\varphi : A \multimap B$ extends a core function $f : \text{Can } c \rightarrow \underline{B}$. Then:

$$\begin{array}{ccccc} \varphi \text{ extends } f & \varphi \text{ is equivariant} & \varphi \text{ extends } f & & \\ \downarrow & \downarrow & \downarrow & & \\ f(g \circledast x) & = \varphi(g \circledast x) & = g \circledast \varphi(x) & = g \circledast fx \end{array}$$

Equivariant where defined core functions

Assume an equivariant $\varphi : A \circledast \rightarrow B$ extends a core function $f : \text{Can } c \rightarrow \underline{B}$. Then:

$$\begin{array}{ccccc} \varphi \text{ extends } f & \varphi \text{ is equivariant} & \varphi \text{ extends } f & & \\ \downarrow & & \downarrow & & \downarrow \\ f(g \circledast x) & = & \varphi(g \circledast x) & = & g \circledast \varphi(x) & = & g \circledast fx \end{array}$$

Let A, B be G -actions and $X \subseteq \underline{A}$. Then, $f : X \rightarrow \underline{B}$ is **equivariant** when

$$\forall g \in \underline{G}. \forall x \in X. (g \overset{A}{\circledast} x \in X \implies f(g \overset{A}{\circledast} x) = g \overset{B}{\circledast} fx)$$

Equivariant where defined core functions

Assume an equivariant $\varphi : A \circledast \rightarrow B$ extends a core function $f : \text{Can } c \rightarrow \underline{B}$. Then:

$$\begin{array}{ccccc} \varphi \text{ extends } f & \varphi \text{ is equivariant} & \varphi \text{ extends } f & & \\ \downarrow & \downarrow & \downarrow & & \\ f(g \circledast x) & = \varphi(g \circledast x) & = g \circledast \varphi(x) & = g \circledast f x \end{array}$$

Let A, B be G -actions and $X \subseteq \underline{A}$. Then, $f : X \rightarrow \underline{B}$ is **equivariant** when

$$\forall g \in \underline{G}. \forall x \in X. (g \overset{A}{\circledast} x \in X \implies f(g \overset{A}{\circledast} x) = g \overset{B}{\circledast} f x)$$

Theorem

Given:

G -actions A, B ; canoniser $c : \underline{A} \rightarrow \underline{G}$; equivariant core function $f : (\text{Can } c \subseteq A) \circledast \rightarrow B$;
Then $\text{wlog}(c, f) : A \circledast \rightarrow B$ is the unique equivariant function extending f .

Simply-typed w.l.o.g.

- ▶ Straightforward to implement
- ▶ Straightforward to use
- ▶ Plumbing group actions with type-classes/interfaces fiddly as usual
- ▶ Plenty of unreachable cases
- ▶ Doesn't cover mathematical reasoning

Straightforward extension to dependent types via indexed families.

Talk structure

- ▶ Introduction
- ▶ Simply-typed symmetric programming
- ▶ Symmetric datatypes à la Marcelo
- ▶ Conclusion

Recap: semantic structure for simply-typed data-types

data types	top-level constructor + sub-structure	example structural induction
$\text{List } A = \text{Nil}$ $\quad \{ (::) \} \times A \times \text{List } A$	$X \mapsto \text{Nil}$ $\quad \{ (::) \} \times A \times X$	length: $\begin{cases} \text{Nil} & \mapsto 0 \\ ((::), a, n) & \mapsto 1 + n \end{cases}$
$\text{Tree } A = \text{Leaf}$ $\quad \{ \text{Node} \} \times \text{Tree } A$ $\quad \quad \times A \times \text{Tree } A$	$X \mapsto \text{Leaf}$ $\quad \{ \text{Node} \} \times X$ $\quad \quad \times A \quad \times X$	depth: $\begin{cases} \text{Leaf} \mapsto 0 \\ \langle \text{Node}, m, x, k \rangle \\ \quad \mapsto 1 + \max \{ m, k \} \end{cases}$

etc. for strictly positive data-types

Recap: semantic structure for simply-typed data-types

General picture:

data type + constructors	top-level constructor + sub-structure	structural induction
	functor $F : \mathbf{Set} \rightarrow \mathbf{Set}$ FX : constructors available using substructures $x \in X$	
initial algebra $\mathbf{roll} : F(\mu F) \xrightarrow{\cong} \mu F$ constructs data from constructor + data for substructures evaluates each constructor homomorphically using algebra structure:		algebra $f : FA \rightarrow A$ evaluate constructor given values for sub-structures
	$\frac{f : FA \rightarrow A}{\mathbf{fold} \ f : (\mu F, \mathbf{roll}) \rightarrow (A, f)}$	F -algebra F -algebra homomorphism

Simply-typed data-types with **symmetry**

Examples

- Pointwise symmetry on lists (G -action A):

$$\boxed{\text{Nil}} \xrightarrow{g \circledast} \boxed{\text{Nil}} \qquad \boxed{a :: x} \xrightarrow{g \circledast} \boxed{(g \circledast a) :: (g \circledast x)}$$

- Tree flipping symmetry (S_2 -action A):

$$\boxed{\text{Leaf}} \xrightarrow{g \circledast} \boxed{\text{Leaf}} \qquad \begin{array}{ccc} & \boxed{n} & \\ x & / \quad \backslash & y \end{array} \xrightarrow{\text{flip} \circledast} \begin{array}{ccc} & \boxed{n} & \\ \text{flip} \circledast y & / \quad \backslash & \text{flip} \circledast x \end{array}$$

Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

[Beck'69]

distributive law for free G -action monad and F , i.e. natural transformation

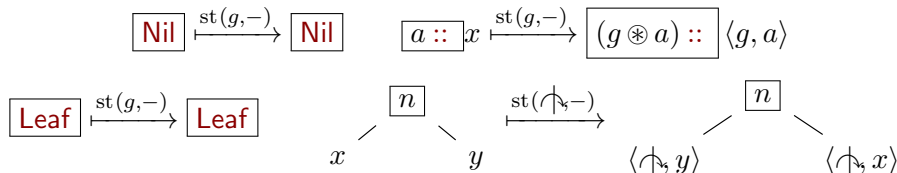
$$\text{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \text{st}_X(e, t) = F(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \text{st}_X(g * h, t) = F(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\text{st}_{\underline{G} \times X}(g, \text{st}_X(h, t)))$$

Examples



Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

[Beck'69]

distributive law for free G -action monad and F , i.e. natural transformation

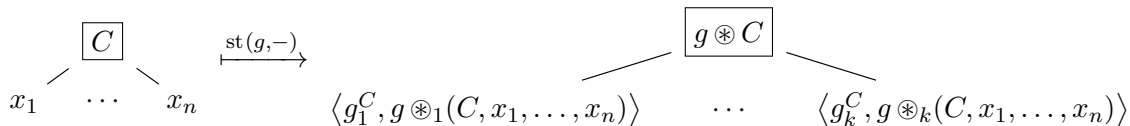
$$\text{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \text{st}_X(e, t) = F(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \text{st}_X(g * h, t) = F(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\text{st}_{\underline{G} \times X}(g, \text{st}_X(h, t)))$$

Generally



Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

[Beck'69]

distributive law for free G -action monad and F , i.e. natural transformation

$$\mathrm{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \mathrm{st}_X(e, t) = F(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \mathrm{st}_X(g * h, t) = F(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\mathrm{st}_{\underline{G} \times X}(g, \mathrm{st}_X(h, t)))$$

Unit law

$$\begin{array}{ccc} \begin{array}{c} \boxed{C} \\ \swarrow \quad \searrow \\ x_1 \quad \cdots \quad x_n \end{array} & \xrightarrow{\mathrm{st}(e, -)} & \begin{array}{c} \boxed{C} \\ \swarrow \quad \searrow \\ \langle e, x_1 \rangle \quad \cdots \quad \langle e, x_n \rangle \end{array} \end{array}$$

Simply-typed data-types with **symmetry**

Symmetry strength (G group, $F : \mathbf{Set} \rightarrow \mathbf{Set}$ functor)

[Beck'69]

distributive law for free G -action monad and F , i.e. natural transformation

$$\text{st}_X : \underline{G} \times FX \rightarrow F(\underline{G} \times X)$$

satisfying:

$$t : FX \vdash \text{st}_X(e, t) = F(x \mapsto \langle e, x \rangle) t$$

$$g, h : \underline{G}, t : FX \vdash \text{st}_X(g * h, t) = F(\langle g', h', x \rangle \mapsto \langle g' * h', x \rangle) (\text{st}_{\underline{G} \times X}(g, \text{st}_X(h, t)))$$

Multiplication law

$$\begin{array}{ccccccc}
 & \boxed{C} & & \boxed{h \otimes C} & & \boxed{(g * h) \otimes C} & \\
 x_1 & \diagdown & \cdots & \xrightarrow{\text{st}(h, -)} & \diagdown & \xrightarrow{F(\langle (*), - \rangle) \circ \text{st}(g, -)} & \diagdown \\
 & \langle h_1^C, y_1 \rangle & & \cdots & & \langle (g * h)_1^C, z_1 \rangle & & \cdots & & \langle (g * h)_k^C, z_k \rangle
 \end{array}$$

Symmetry strengths lift actions

Proposition (lifted action $F_{\text{st}}A = \langle F\underline{A}, (\overset{\text{st}}{*}) \rangle$, cf. [Beck '69])

$$\frac{G\text{-action } (*) : \underline{G} \times \underline{A} \rightarrow \underline{A}}{G\text{-action } (\overset{\text{st}}{*}) : \underline{G} \times F\underline{A} \xrightarrow{\text{st}} F(\underline{G} \times \underline{A}) \xrightarrow{F(*)} F\underline{A}}$$

Moreover:

$$\frac{h : A \overset{*}{\rightarrow} B \quad \text{equivariant}}{Fh : F_{\text{st}}A \overset{*}{\rightarrow} F_{\text{st}}B \quad \text{equivariant}}$$

Lifted action

Examples

- Pointwise symmetry on lists (G -action A):

$$\boxed{\text{Nil}} \xrightarrow{g \circledast^{\text{st}}} \boxed{\text{Nil}} \qquad \boxed{a :: x} \xrightarrow{g \circledast^{\text{st}}} \boxed{(g \circledast a) :: (g \circledast x)}$$

- Tree flipping symmetry (S_2 -action A):

$$\boxed{\text{Leaf}} \xrightarrow{g \circledast^{\text{st}}} \boxed{\text{Leaf}} \qquad \begin{array}{c} \boxed{n} \\ \swarrow \quad \searrow \\ x \qquad y \end{array} \xrightarrow{\updownarrow \circledast^{\text{st}}} \begin{array}{c} \boxed{n} \\ \swarrow \quad \searrow \\ \updownarrow \circledast y \quad \updownarrow \circledast x \end{array}$$

Equivariant iterators

Equivariant $\langle F, \text{st} \rangle$ -algebra $\langle \underline{A}, (\otimes), f \rangle$

- ▶ G -action $A = \langle \underline{A}, (\otimes) \rangle$
- ▶ equivariant F -algebra $f : F_{\text{st}} A \otimes \rightarrow A$, i.e., an F -algebra $f : F \underline{A} \rightarrow \underline{A}$ such that

$$f(g \overset{F_{\text{st}} A}{\otimes} t) = g \overset{A}{\otimes} f(t) \quad \begin{array}{ccccc} \underline{G} \times F \underline{A} & \xrightarrow{\text{st}} & F(\underline{G} \times \underline{A}) & \xrightarrow{F(\otimes)} & F \underline{A} \\ \text{id} \times f \downarrow & & = & & \downarrow f \\ \underline{G} \times \underline{A} & \xrightarrow{\quad\quad\quad} & & & \underline{A} \\ & & (\otimes) & & \end{array}$$

Equivariant algebra homomorphism $h : \langle A, f \rangle \otimes \rightarrow \langle B, g \rangle$

simultaneously:

- ▶ an F -homomorphism
- ▶ equivariant map $h : A \otimes \rightarrow B$

Symmetric datatype action

Theorem (equivariant induction principle)

Let $\text{roll} : F\mu F \rightarrow \mu F$ be an initial algebra for F with G -strength st . Defining

$$(\circledast) := \text{pfold roll} : \underline{G} \times \mu F \rightarrow \mu F$$

makes $\langle \mu F, \text{roll} \rangle$ into the **initial** equivariant $\langle F, \text{st} \rangle$ -algebra:

$$\frac{\text{equivariant algebra } f : F_{\text{st}} A \circledast \rightarrow A}{\text{equivariant } F\text{-homomorphism } \text{fold } f : \mu F \circledast \rightarrow A}$$

Theorem (parameterised initiality [Bird and Paterson'99])

For every F -algebra $f : FX \rightarrow X$ there is a unique **parameterised homomorphism**:

$$\text{pfold } f : \underline{G} \times \mu F \rightarrow X \quad \text{satisfying} \quad \text{pfold } f \langle g, \text{roll } t \rangle = f(F(\text{st}(g, t)))$$

Symmetric datatype action

Theorem (equivariant induction principle)

Let $\text{roll} : F\mu F \rightarrow \mu F$ be an initial algebra for F with G -strength st . Defining

$$(\circledast) := \text{pfold roll} : \underline{G} \times \mu F \rightarrow \mu F$$

makes $\langle \mu F, \text{roll} \rangle$ into the **initial** equivariant $\langle F, \text{st} \rangle$ -algebra:

$$\frac{\text{equivariant algebra } f : F_{\text{st}} A \circledast \rightarrow A}{\text{equivariant } F\text{-homomorphism } \text{fold } f : \mu F \circledast \rightarrow A}$$

(Because the free G -action functor has a right-adjoint $(\underline{G} \times -) \dashv (\underline{G} \rightarrow -)$)

Symmetric datatype action

Example: Pointwise action on lists

$$g \circledast (x_1 :: \cdots :: x_n :: \text{Nil}) = (g \circledast x_1 :: \cdots :: g \circledast x_n :: \text{Nil})$$

Example: Flipping the tree

$$\mathbf{F} \circledast \left(\begin{array}{c} & & 3 \\ & \swarrow & \searrow \\ 1 & 2 & 4 \\ \swarrow & & \searrow \end{array} \right) = \left(\begin{array}{c} 3 & & \\ \swarrow & & \searrow \\ 4 & 2 & 1 \\ \swarrow & & \searrow \end{array} \right)$$

Wlog for symmetric datatypes

Canoniser for G -symmetric functor

Natural transformation:

$$c_X : FX \rightarrow \underline{G}$$

Canonical elements functor

$$\text{Can } c : G\text{-Act} \rightarrow \text{Set} \quad (\text{Can } c)A := \{(cu) \circledast u \mid u \in F\underline{A}\}$$

NB: $(\text{Can } c)A = \text{Can } c_{\underline{A}}$

Core algebra $f : (\text{Can } c)A \rightarrow \underline{A}$

equivariant when $f : (\text{Can } cA \subseteq F_{\text{st}}A) \circledast \rightarrow A$

Folded wlog construction

$$\frac{G\text{-strong } F \quad G\text{-action } A \quad c : F \rightarrow \underline{G} \quad f : (\text{Can } c)A \rightarrow \underline{A}}{\mathbf{flog}(c, f) := \mathbf{fold}(\mathbf{wlog}(c_{\underline{A}}, f)) : \mu F \rightarrow \underline{B}}$$

$\mathbf{flog}(c, f)$ equivariant iff f is.

Indexed datatypes

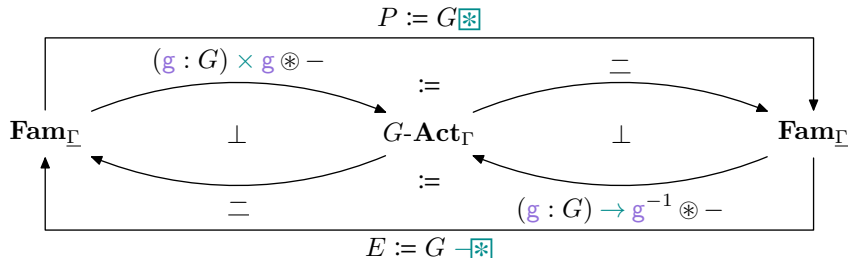
Whole theory generalizes to dependent types since:

$$G\text{-}\mathbf{Act}_\Gamma \simeq \widehat{\oint \Gamma} \quad \mathbf{Fam}_\Gamma \simeq \widehat{\oint \Gamma} = \left| \widehat{\oint \Gamma} \right|$$

Where:

- ▶ $\oint F$: Grothendieck construction $((F : \mathbb{C}^{\text{op}} \rightarrow \mathbf{Set}) \in \hat{\mathbb{C}} \text{ presheaf})$;
- ▶ $|\mathcal{C}|$: discrete category underlying category $\mathcal{C}$

Thanks to Marcelo [Fiore-Menni'05] we have monadic adjunctions:



Proof-of-concept: sub-tree projection in Idris 2 [Brady'21]

Non-indexed datatype—trees:

```
data BinTree a = Leaf | Node (BinTree a) a (BinTree a)
```

Indexed datatype—path in a tree:

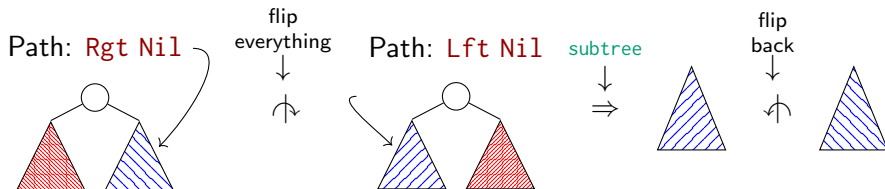
```
data Path : BinTree a -> Type where
```

```
  Nil : Path (Node l k r)
```

```
  Lft : Path l -> Path (Node l k r)
```

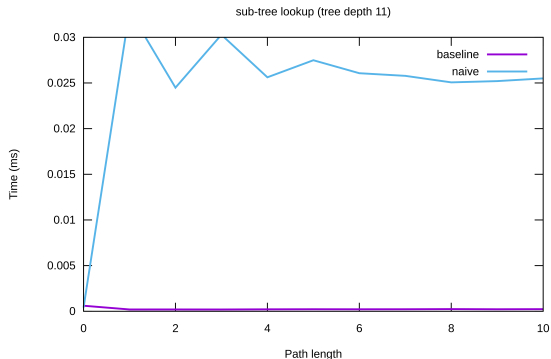
```
  Rgt : Path r -> Path (Node l k r)
```

```
subtree : {t : BinTree a} -> Path t -> BinTree a
```



Proof-of-concept: sub-tree projection in Idris 2 [Brady'21]

- ▶ Baseline implementation: include the symmetric—redundant—case (**Rgt**)
- ▶ Naive implementation: use **flog**



Recipe 1: algebraic secret sauce

To optimise an expensive repetitive operation:

- ▶ work out the (generalised) algebra $A \models \mathcal{T}$ of the operation
- ▶ carry the computation out in $\mathbf{Free}_{\mathcal{T}} \underline{A}$
- ▶ evaluate in algebra to return to $\underline{A}$

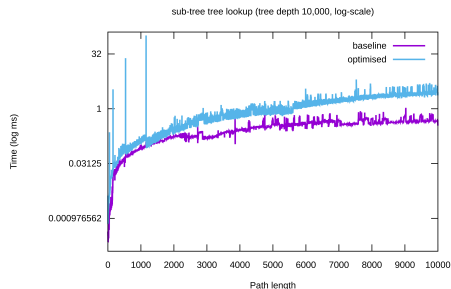
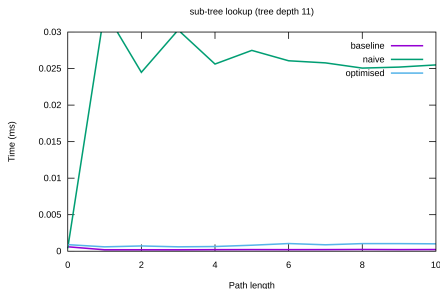
Examples galore, e.g.: choose basis for vector space to make it free.

Recipe 2: symmetric strength for auxiliary datatypes

For auxiliary datatypes (i.e., not directly involved in the induction):
use symmetric strength directly.

Proof-of-concept: sub-tree projection in Idris 2 [Brady'21]

- ▶ Baseline implementation: include the symmetric—redundant—case (**Rgt**)
- ▶ Naive implementation: use **flog**
- ▶ Optimised implementation: use **flog**, free action, strength for auxiliary data



Happy birthday Marcelo — many happy years longer!